Occam — アルゴリズム解剖書

2026-10-12 ／ 全モジュールの機構・定数・不変条件

> **すべて実装から転記。**定数・閾値・正規表現は出典の行から取っている。実装とずれたら実装が正しい。

0. 読み方

記号	意味
[LOCK]	KeepOut が AST レベルで封じている領域。整形・コメント・変数改名は通るが、構造変更は拒否される
不変条件 n	§1 の3つのうち n 番目
〔製品〕	`py/occam.py` から推移的に到達する経路
〔計測〕	確定値を作った経路。製品からは呼ばれない
〔降格〕	候補生成。`--emit-molecules` なしでは止まる

本書の対は `docs/final_report.md`（何を establish したか）。

本書は**どう動くか**だけを書く。

1. 不変条件（3つ）

すべて `engine/` の Rust にあり、KeepOut 9ロックのうち5つがこれを守っている。

#	主張	強制の方法	場所
1	carry（オペレータ間を生き延びる唯一の値）の長さが固定。バルクサイズにもデータ量にも依存しない	`#[derive(Clone, Copy)]` が `Vec` / `HashMap` / `Box` / `String` を型として排除。`const` assert がサイズを固定	`carry.rs` [LOCK]
2	1ファイル1行グループ。parquet のフッタは行グループ数に比例するので、複数持つとメモリがデータ量に依存する	読み込み時に `num_row_groups() > 1` なら**エラーで拒否**	`bulk.rs` [LOCK]
3	バルクと演算器を同時に1つしか持たない	`AtomicUsize` の `compare_exchange(0, 1)`。2つ目の生成で panic	`bulk.rs` [LOCK] / `process.rs` [LOCK]

不変条件3 の核心

```
// engine/src/main.rs [LOCK: ast name="one_bulk_per_iteration"]
for bulk in &mut bulks {
    let bulk = bulk?;
    let mut op = DomainOperator::new(pass, args.learning_rate);
    carry = op.absorb(&bulk, carry);
    // op and bulk are dropped here, before the next iteration
}
```

`let mut op = ...` を**再代入**にすると、次の演算器を**前の演算器を drop する前に**

構築する。`compare_exchange` がそれを捕まえる（禁忌6）。

検査の実体

```
// bulk.rs — compare_exchange であって fetch_add ではない。
// 検査と確保が1ステップでなければならない。
fn new(rows: Vec<Sample>) -> Self {
    let claimed = LIVE_BULKS.compare_exchange(0, 1, Ordering::SeqCst, Ordering::SeqCst);
    assert!(claimed.is_ok(), "two bulks alive at once");
    Bulk { rows }
}
```

2. エンジン（Rust、5ファイル）〔製品 + 計測〕

製品経路から呼ばれる。`py/screenable.py:63` の `ENGINE_FP` が

`engine/target-fp/release/engine` を subprocess で起動し、順位付けの可否を判定する。

2.1 `carry.rs`（425行）— 固定長の状態

定数

定数	値	役割
`K`	**256**	候補スロット数。コンパイル時固定
`F`（既定ビルド）	**9**	入力特徴数（物理化学記述子）
`F`（`--features fingerprint`）	**2048**	指紋ビット数
`STANDARDIZE`	`!fingerprint`	連続値は標準化、疎な二値は**しない**
`QUADRATIC`	`!fingerprint && !linear`	2次項（平方と対積）を足すか
`W`	`QUADRATIC ? F + F + F(F−1)/2 : F`	モデル次元。F=9 なら **54**
`BINS`	**1024**	スコアヒストグラムのビン数（クラスごと）

なぜ疎な二値を標準化しないか（コメントから）

> 割合 p のビットは分散 p(1−p) を持つので、標準偏差で割ると稀なビットが

> 1/√(p(1−p)) 倍される。p = 0.001 では立ったビットが **+31** になり、

> 内積が少数の稀なビットで決まる。平均を引くと疎性も壊れ、**全ての0ビットが −p** に

> なってベクトルが密になる。

`Carry` の中身

```
pub struct Carry {
    pub top: [Candidate; K],        // 候補スロット
    pub bulks_seen: u64,
    pub weights: [f32; W],          // pass 1 で学習
    pub bias: f32,
    pub mean: [f32; F],             // py/descriptors.py 由来。ここでは学習しない
    pub inv_std: [f32; F],
    pub seen: u64,                  // 学習率スケジュールを駆動
    pub hist: [[u32; BINS]; 2],     // 保留行のスコア、クラス別
    pub prequential: [[u32; BINS]; 2],
}
```

`Copy` が不変条件1を強制する。ヒープを持つフィールドを足すと derive が失敗し、

下の `const` assert も合わなくなる。

```
const PAYLOAD_BYTES: usize = K * size_of::<Candidate>()
    + 2 * size_of::<u64>()
    + (W + 2 * F + 1) * size_of::<f32>()
    + 4 * BINS * size_of::<u32>();
const _: () = assert!(CARRY_BYTES >= PAYLOAD_BYTES - 8
                   && CARRY_BYTES <= PAYLOAD_BYTES + 8);
```

Rust はフィールドを並べ替えるので、**厳密な合計ではなく「アラインメント以上の

パディングが無い」**という形で縛っている。

固定長状態から大域 AUC を出す

これが鍵。スコアを [0,1] に押し込んで 1024 ビンのヒストグラムに入れ、

そこから AUC を計算する。**全スコアを保持しない。**

```
pub fn auc(hist: &[[u32; BINS]; 2]) -> Option<f64> {
    let neg = hist[0].iter().sum(); let pos = hist[1].iter().sum();
    if neg == 0 || pos == 0 { return None; }
    let mut seen_neg = 0u64; let mut concordant = 0f64;
    for bin in 0..BINS {
        let n = hist[0][bin]; let p = hist[1][bin];
        // このビンの陽性は下の全陰性に勝ち、同ビンの n 件と引き分ける
        concordant += p as f64 * (seen_neg as f64 + n as f64 / 2.0);
        seen_neg += n;
    }
    Some(concordant / (pos as f64 * neg as f64))
}
```

同ビン内の同点は**半分ずつ分ける**（台形則が同点ブロックに対して行うのと同じ）。

厳密 AUC との差は実測で **8.9×10^{-5}**。

順位付けは logit で行う（禁忌14）

```
pub fn logit(&self, x: &[f32; W]) -> f32   // ← 順位付けはこれ
pub fn predict(&self, x: &[f32; W]) -> f32 // = sigmoid(logit)
```

> ロジスティック関数は単調なので厳密演算では順序を保つが、**f32 では飽和する。**

> モデルが適合していないデータでは z が大きくなり、`sigmoid(z)` が多数の行で

> **ちょうど 1.0 に丸まる。**そこでの順序は失われ、安定ソートが黙って入力順に

> 退化する ― **スコアで選んでいるように見えて、ファイル位置で選んでいる。**

M3 の2回目の測定が撤回された原因がこれ。

候補スロットの更新

```
pub fn credit(&mut self, id: u64, score: f32) {
    // 既知の id は support を +1、score は max を取る
    // 未知の id は空きスロットを取るか、最弱スロットより高ければ置換
}
```

`Candidate::EMPTY` は `score: f32::NEG_INFINITY, support: 0`。

空きスロットは常に実候補より弱い（`weaker_than` が明示的に扱う）。

2.2 `bulk.rs`（270行）― 読み手

不変条件3	**`LIVE_BULKS` の `compare_exchange`。返したバルクへの参照を保持せず、先読みしない**
不変条件2	`open_part()` が `num_row_groups() > 1` を拒否。**M3 で実際に発火した**

glibc の mmap 閾値を固定する（禁忌5）

```
const M_MMAP_THRESHOLD: c_int = -3;
unsafe { mallopt(M_MMAP_THRESHOLD, 64 * 1024); }
```

> 64 KiB 以上の確保を mmap にすると、drop したバルクとそのデコード済みページが

> OS に返る。設定しないと glibc が最初の free 後に閾値を上げ、以降のバルクを

> ヒープから供給してフラグメントし、**ピークがバルク数とアドレス配置で揺れる**

> （実測で最大 **+19%**）。明示設定は glibc の動的調整も無効にする。

2.3 `process.rs`（180行）— 演算器

定数	値
`DEFAULT_LEARNING_RATE`	**0.5**（validation で選定、test では選ばない）
`L2`	**1e-5**（サンプルごと）

1サンプル分の SGD

```
let x = Carry::expand(&carry.standardize(&sample.features));
let p = carry.predict(&x);
Carry::record(&mut carry.prequential, p, sample.label); // 学習する前のスコア
carry.seen += 1;
let rate = self.learning_rate / (carry.seen as f32).sqrt();  // 1/√seen 減衰
let error = p - sample.label as f32;
for i in 0..x.len() {
    carry.weights[i] -= rate * (error * x[i] + L2 * carry.weights[i]);
}
carry.bias -= rate * error;
```

2パス必要な理由

パス	内容
Learn	訓練行から重みを更新。同時に prequential ヒストグラムを埋める（**学習する前**のスコア）
Score	重みを凍結し、保留行を carry のヒストグラムに入れる

> 学習しながらスコアを取ると、**あるモデルのスコアと別のモデルのスコアが混ざり**、

> 出てきた数字がどのモデルのものでもなくなる。

2.4 `main.rs`（244行）— ループだけ

```
usage: engine <descriptor-dir> [--stats stats.json] [--bulk-size N]
              [--evaluate train|valid|test] [--epochs N] [--learning-rate R]
              [--dump-scores FILE] [--save-weights FILE] [--load-weights FILE]
```

パス間を生き延びるのは carry だけ。

2.5 `metrics.rs`（32行）— ピーク RSS

`/proc/self/status` の **VmHWM** を、ループ終了**後に1回だけ**読む。

処理経路の中では何も走らない。`key=value` 1行を stderr に出し、

`tests/test_memory_invariant.py` が解析する。

`ru_maxrss` は使えない（禁忌2）。subprocess は vfork なので親のメモリを引き継ぐ。

3. アシスト経路〔製品〕

```
occam.py ─┬→ provenance.py    ライブラリの出自とルール照合
          ├→ knobs.py         回せる条件と出自の型
          ├→ sweep.py         K仕様を1パスで評価
          ├→ propose.py       制約充足 + 最小緩和 + frontier
          ├→ feasible.py      4値判定と必要件数
          ├→ inventory.py     証拠の棚卸し（ファイルから読む）
          ├→ screenable.py    順位付けの可否・適用範囲 → Rust エンジン起動
          ├→ what_to_measure.py  必要な測定件数
          └→ render.py        表示層（識別子を画面に出さない）
```

3.1 `occam.py`（約700行）— 単一の入口

散文の読み取りは閉じた語彙への決定的照合。LLM を使わない

> 言語モデルは仕様に翻訳できるが、**その読み取りは検査できない。**しかも

> **助けようとする誘因を持つ唯一の部品**になる — 不可能な標的を求められたら

> 近いものを見つけてしまう。これはこの系が防ぐために存在する失敗そのもの。

成立するのは**語彙が閉じているから**。4つの性質、10の設定、1つの件数。

読み取りの順序（`parse()`）

順	何を	消費
1	**導入句の強制語**（`^[^:]{0,160}:` 内の `must` 等）	スパンを消費。**以降の全要件が strict**
2	**事実の主張**（FACT_CLAIMS）	消費して `dropped_claims` へ
3	**ライブラリ**（LIBRARIES、先頭一致で break）	消費
4	**成果物**（DELIVERABLE）	消費
5	**語彙の条件**（VOCABULARY）	消費。強制マーカーのスパンも消費
6	**数値の条件**（NUMERIC）	消費。登録外なら `refused`
7	**電荷語**（CHARGE_WORDS、**消費済みスパンの外のみ**）	消費して `refused`
8	**標的**（TARGET_SHAPES）	消費
9	残り = `unread`	—

7の「消費済みスパンの外のみ」が要点。`neutral ZINC` は3でライブラリとして `neutral` を消費済みなので、7は発火しない。独立した条件として書いた場合だけ止まる。

定数

定数	値			
`STRICT_WINDOW`	**40**（強制マーカーが効く範囲。カンマで切れる）			
`LEAD_IN`	`^[^:]{0,160}:`			
`STRICT_MARKERS`	`strictly` `must` `hard` `required` `no more than` `non-negotiable` `exactly` `fixed` `do not relax` `keep`			
`DELIVERABLE`	`(\d[\d,]*)\s+(unique	distinct	different)?\s* (scaffolds?	骨格)` ほか

`骨格` は**入力語彙**（受け付ける側）。出力はすべて英語。

挿入語の許容（禁忌71）

```
_FILLER = r"(?:\s+(?:must|should|shall|is|are|be|being|to|the|a|an|kept|held|
             strictly|exactly|hard|required|fixed|no)\b)*"
_COMPARATOR = r"(?:\s*(?:(?:at\s+most|at\s+maximum|no\s+more\s+than|up\s+to|
               under|below|less\s+than|fewer\s+than|of)\b|<=|<|≤|=))?"
```

初版は `rotatable bonds at most 5` しか読めず、`the rotatable bonds must be at most 5` が**黙って未読リストに落ちていた。**ここに挙がる語は意味を持たない（コピュラ・冠詞・強制マーカー）。強制マーカーは別経路で読む。

停止するもの（すべて exit 1）

条件	メッセージ
ライブラリが公表ルールと矛盾	`the library contradicts the published rule the request is built on`
条件が別の障壁の規則	`the conditions asked for are a published rule about a different barrier`
語彙外の値・量	`the request named something this cannot read as a requirement`
ライブラリ無し	`no library named.`
成果物無し	`no deliverable named.`
`--out-dir` が作れない	`cannot write to --out-dir ...`

3.2 `knobs.py`（約340行）— registry

出自の型

型	意味
`published`	公表規則が既に言っている範囲内。新しい化学的主張をしない
`relaxation`	数値閾値を動かしただけ
`unjustified`	新しい化学的主張。**提案禁止**

設定（`ORDER = tuple(KNOBS)` で導出）

knob	設定	限界値	出自
nitrogen	`primary_amine`	SMARTS	published（Richter 2017「unhindered primary amine best」）
	`primary_or_secondary_amine`	SMARTS	published
	`basic_nitrogen`	SMARTS	published（同条件の最広義）
rotatable	`rotatable_le_5`	5	published（eNTRy 第3条件）
	`rotatable_le_7`	7	relaxation
	`rotatable_le_9`	9	relaxation
weight	`mw_under_450`	450.0	relaxation（本研究の選択）
	`mw_under_600`	600.0	**published（二次：Richter 2017 内の逐語引用）**
	`mw_unbounded`	inf	published

SMARTS

```
ENTRY_PRIMARY_AMINE = [$([N;H2;X3][CX4]),$([N;H3;X4+][CX4])]
SECONDARY_AMINE     = [$([N;H1;X3]([CX4])[CX4]),$([N;H2;X4+]([CX4])[CX4])]
BASIC_NITROGEN      = [$([NX3;H0,H1,H2;!$(N[!#6]);!$(N*=[O,N,S]);!$(N#*);
                         !$([N][a])]),$([NX4+;!$(N*=[O,N,S])])]
```

適用障壁（`APPLIES_TO`）

この registry はグラム陰性外膜についてのもの。記録されるまで、どこにも

書いていなかった。血液脳関門のデータに向けても最後まで走って何も報告しなかった。

引退（`RETIRED`）と不可（`BLOCKED`）

	内容
`RETIRED['charge']`	**描画上の形式電荷**を読んでいた。ZINC は**予測プロトン化状態**で分けている。**別の量が同じ語を共有していた**
`BLOCKED['library_charge_slice']`	スライスは knob ではなくダウンロード時の選択。一級アミン含有率 0.21% / 2.46% / 13.06%
`BLOCKED['target_identity']`	標的を記録したデータが無い。全ラベルが二値
`BLOCKED['globularity']`	3D 配座が必要。**実装済みだが 300件サンプルで 100% 通過、中央値 0.075、最大 0.247（閾値 0.25）**

引退した knob を含む旧レコードは `knobs.Retired` 例外で拒否する。

黙って knob を落とすと、記録が別の screening の記述に変わる。

3.3 `sweep.py`（約310行）— K仕様を1パス

HyperLogLog

定数	値
`HLL_P`	**12**
`HLL_M`	**4096** = 1 << 12
`HLL_ALPHA`	0.7213 / (1 + 1.079/4096)
状態	**4,152 B / 仕様**（レジスタ 4,096 B + α）

```
digest = hashlib.blake2b(text.encode(), digest_size=8).digest()
value = int.from_bytes(digest, "big")
index = value >> (64 - HLL_P)          # 上位12ビットでレジスタ選択
rest  = (value << HLL_P) & ((1 << 64) - 1)
rank  = 1 if rest == 0 else 64 - rest.bit_length() + 1
if rank > self.registers[index]: self.registers[index] = rank
```

レジスタに空きがあるうちは**線形計数**が正確な推定量なので、

`raw <= 2.5 * HLL_M and zeros` で切り替える。

Welford（`Running`）

平均と分散を値を保持せずに更新する。MW と回転結合の分布に使う。

1パスの構造

```
for text in iter_library(part_dir, limit):
    mol = smiles_mod.parse(text)
    weight    = Descriptors.MolWt(mol)
    rotatable = rdMolDescriptors.CalcNumRotatableBonds(mol)
    hits = {name: bool(body["test"](mol)) for name, body in nitrogen_settings.items()}
    # scaffold は遅延。生存者だけが払う
    for variant in variants:
        variant.consider(mol, weight, rotatable, scaffold_of, hits)
```

scaffold 計算は分子あたりコストの 27.6% を占め、生存者は測定スライスの 2.5% しか

いなかった。**全分子で計算すると実行の4分の1を、どの仕様も残さない行に使う。**

最初に要求した variant まで遅延する。

変種の列挙

`enumerate_variants(baseline, only_looser=True)` は `knobs_mod.ORDER` から

直積を取り、**baseline より厳しい組み合わせを除外**する。

3 knob × 3 設定 = **27**（charge 引退前は 54）。

3.4 `propose.py`（約320行）— 提案

定数	値
`SKETCH_RELATIVE_ERROR`	**0.016**（2^12 レジスタでの標準誤差。本リポジトリで4基数について 0.29%〜1.24% と実測）
`UNCERTAIN_BAND`	**0.048** = 3 × 0.016
`FRONTIER`	**8**

3値分類（sketch のみ）

```
estimate >= required × (1 + 0.048)  → feasible
estimate <= required × (1 - 0.048)  → infeasible
それ以外                             → uncertain
```

正確カウントの対象

	なぜ
uncertain 帯の全仕様	sketch が構成上決められない
frontier に出る全仕様（上限8）	人が trade を比較する数字。**2行の間の 1.6% の差は差として読まれる**
推奨される勝者	—

3σ は確率的保証で、27仕様あると1回の実行でどれかが帯の外に出る確率は無視できない。

frontier を無条件に数えることで、読み手が見る数字から帯が消える。

1仕様ずつ、都度捨てる（禁忌84）

frontier 8件は「正確パスの回数」の上限で、「状態量」の上限ではない。

8セットを同時に持つと `8 × 46,437骨格 × 127.4 B ≈ 45 MB` で、sketch が避けている額と

ほぼ同じ。逐次実行でピークは最大の1仕様分（**約 5.6 MiB**）。

`exact_scaffold_count` は**件数だけを返す**ので、呼び出し側が集合を蓄積できない。

スコープ検査（禁忌85）

```
sweep が読んだ limit ≠ 正確パスの limit  → 停止
骨格数 > 生存数                          → 落ちる
```

後者は**このクラス全体を捕まえる最も安い検査。**

順位（重み付きスコアを使わない）

```
def rank(row):
    return (row["steps_from_baseline"],
            0 if row["provenance"]["weakest"] == "published" else 1,
            -row["survivors"])
```

辞書式のみ。第3項は決定的タイブレークであって選好ではない。

3.5 `provenance.py`（約280行）— ライブラリは入力

スライス	**予測電荷**
N	0（中性）
O	+1
M	−1
P	+2 以上

```
_SLICE = re.compile(r"-([NOMP])(?:\.[a-z])?\.smi\.gz$")
```

世代の文字（`.x`）を許容する形。改名で世代を落とすと、出自を読む仕組みが

自分の改名で動かなくなる（禁忌67）。

矛盾検査の根拠（Richter 2017 の逐語値）

```
ENTRY_ACCUMULATION_BY_CHARGE = {
    "positive": {"accumulated": 12, "total": 41},
    "neutral":  {"accumulated":  0, "total": 39},
    "negative": {"accumulated":  0, "total": 20},
}
```

inert 検出

ある knob だけが違う仕様ペアを全部比べ、生存数が全組で同一なら inert。

`charge` はこの run で 27/27 同一だった。

> **何も変えない knob を制約として報告してはいけない。**読み手はフィルタが

> 走ったと信じる。

3.6 `screenable.py`（約640行）— 何が establish できるか

定数	値
`MIN_DISTINCT_SHARE`	**0.50**
`MAX_TIE_SHARE`	**0.05**
`EXTRAPOLATION_BELOW`	**0.40**
`SAMPLE`	**1200**
`SAMPLE_SEED`	**0**

閾値は導出していない。最近傍 Tanimoto の 0.4〜0.6 が実務の慣行で、

0.4 はその下端。**集合を「内側」と言う方向に誤るように選んで、書き留めた。**

rankability — ここで Rust エンジンが起動する

1. 生存者を seed 0 でサンプル（50件未満なら `not_measurable`）

2. 指紋（Morgan radius 4、2048 ビット）を numpy ブロックに

3. 一時ディレクトリに parquet パートを書き、`stats.json` を作る

4. **`engine/target-fp/release/engine` を subprocess 起動**

5. スコアの distinct 率と同点率から判定

applicability

各生存者の**訓練集合への最近傍 Tanimoto**を取り、中央値が 0.4 以上なら

`answerable`、未満なら `answerable_but_extrapolating`。

対照（`CONTROLS`）

対照	測定	結果
`tanimoto_pipeline`	IMI-COMBINE から300件を抜き、残り21,752 への最大 Tanimoto	中央値 **0.747**、p5 0.358、最大 1.000、閾値以上 **94.0%** → **pass**
`globularity`	GramNegAccum table4 の68件を著者の `glob` 列と比較	r = **0.529**、中央値 0.156 対 0.181、範囲 [0.037, 0.343] 対 [0.046, 0.450]、閾値一致 82.4% → **「範囲については pass、分子単位では弱い」**

globularity の対照には留保が明記されている — 12/68 が 0.25 閾値で不一致、

配座で最大 0.236 動く。**分布に使い、単一化合物には使わない。**

3.7 `what_to_measure.py`（約420行）— 拒否の次の一歩

定数	値
`COVERED_AT`	**0.40**（screenable の適用範囲閾値と同じ）
`BAND`	(0.30, 0.40, 0.50, 0.60)
`MAX_MOLECULES`	**6000**（Butina は全距離行列を保持。禁忌11）
`SAMPLE_SEED`	**0**

変換

```
適用範囲の検定     各化合物に閾値内の訓練近傍があるか
        ↓ 同じ閾値で
クラスタリング     近傍を持たない領域 = 1クラスタ
        ↓
出力                  クラスタ数 = 測るべき化合物数
```

Butina は距離で切り、適用範囲は類似度で判定する。同じ 0.4 が逆向きなので、

`cutoff = 1.0 - covered_at` = **0.6** で切らないと、適用範囲の検定が既に

「覆われている」と見なす近傍を数えてしまい、**買う化合物数がおよそ倍になる。**

出力に必ず付くもの

— 計算した指紋の名前（`Morgan radius 4, 2048 bits (ECFP8)`）

— 閾値感度（帯での変動倍率）

— **標的が無いこと**（ヒットセットではなくカタログの部分集合）

— **代表化合物を選んでいないこと**と、選ぶなら Butina 重心であること

— **被覆の要件であって精度の約束ではない**こと

3.8 `feasible.py`（約270行）— 4値判定

定数	値
`EXTRAPOLATION_BELOW`	**0.40**
`SAMPLE`	**3000**

```
answerable                      中央値 >= 0.40
answerable_but_extrapolating    中央値 <  0.40
not_answerable                  訓練構造が読めない / ライブラリが読めない
undecidable                     （label_available 経由）
```

必要件数（半幅 1/√n）

```
n = observed_n * (observed_half_width / margin) ** 2
```

禁忌56：**必要なデータ量は「問い」を決めないと出ない。**同じ式から、偶然超えを

言うには約150、2方向の区別には別の数が出る。

SMARTS は registry から読む（2026-10-11 に修正）

降格済みの `candidates.py` から引いていた。**一級アミンの定義が2箇所にあった**

（禁忌31 と同型）。限界値 5 / 450 / 600 も registry から読む。

3.9 `inventory.py`（220行）— 証拠をファイルから読む

定数	値
`MAX_LABEL_CARDINALITY`	**24**（これ以下の distinct 値を持つ文字列列をラベルと見なす）
`SAMPLE_ROWS`	**20000**
`STRUCTURE_COLUMNS`	`smiles` `mol` `inchikey` `skeleton` `scaffold` `zinc_id`

> **データセットの名前も文書中の記述も、中身の証拠として受け取らない。**

> IMI-COMBINE がまさにそれで、活性モデルが長らく「結合モデル」と記述された後に、

> **実際に見たら標的の記録が無い二値ラベルだった。**

基数が有用な信号。distinct 値が数個の文字列列はラベル、行数と同数なら識別子か構造。

二値のラベル列は、データセットが何と呼ばれていようと標的を解決できない。

3.10 `render.py`（約390行）— 表示層

識別子を画面に出さない。記録（JSON）は識別子のまま。

	画面	記録
	`under 450 Da`	`mw_under_450`
	`an unhindered primary amine`	`primary_amine`
	`ZINC22 tranche H29P240, -0 slice (+1 at pH 7.4), 636,669 compounds`	`data/work/zinc_0`
	`cannot be answered`	`not_answerable`

文言が無い設定は推測せず識別子のまま出す。綴りから組み立てた英文は、

数字を間違えていても流暢に読めてしまう。

`tests/test_render.py`（26本）が**文言中の数字を registry の `limit` と照合**する。

定数	値
`WIDTH`	**78**（折り返し幅）

4. 計測・検証〔計測〕

製品からは呼ばれない。§4 の確定値を作った経路で、記録として残っている。

4.1 `folds.py`（303行）— 分割

定数	値
`PART_ROWS`	8192（パート行数 **かつ** `row_group_size`）
`MORGAN_RADIUS`	**2**
`MORGAN_BITS`	2048
`BUTINA_CUTOFF`	**0.4**（Tanimoto **距離**）
`BUTINA_MAX_MOLECULES`	**6000**（超えたら拒否）

分割はグループ単位で、行単位では絶対に行わない。

> scaffold が fold をまたがない。**fold の半分より大きい scaffold を先に配置**するので、

> どの fold に入るかを支配できない。

Butina を使う理由（禁忌12）：CHAMP では上位2 scaffold（ベンゼン単環・環なし）が

52.7% を占め、fold に分けられない。指紋でクラスタリングすると

グループサイズに上限がかかる。

O(n²) の距離行列を保持するので 6,000件超で OOM（禁忌11）。拒否して理由を言う。

seed は1つの整数で `random.Random` を駆動する。`folds.py --repeat` がそれ。

4.2 `cv.py`（237行）— 反復 k-fold CV

定数	値
`ENGINE`	`OCCAM_ENGINE` 環境変数で上書き可
`LEARNING_RATES`	**(0.05, 0.1, 0.25, 0.5, 1.0, 2.0)** — validation fold で掃引する6候補

AUC（`exact_auc`）

midrank Mann–Whitney。同点ランは `average_rank = (i + j) / 2 + 1`。

CI（`bootstrap_ci(values, rounds=20000, seed=0)`）

非パラメトリックのパーセンタイル・ブートストラップ。ただし

行ではなく反復の平均に対して取る。

seed の二重の役割

> `repeat ∈ range(args.repeats)` が**同時に** `folds_mod.build` に渡す

> fold 割り当ての seed であり、sklearn ベースラインの `random_state` でもある。

4.3 `bench.py`（183行）— 21 seed の AUC

既定	値
`--seeds`	**21**
`--epochs`	**1**
`--bulk-size`	**1024**
`--evaluate`	`test`

**8:1:1 ホールドアウトを多数の seed で回す。記述子は1回だけ計算し、seed ごとに

分割だけ引き直す。**

不確かさは平均 ± 標本標準偏差（n−1）のみ。ブートストラップも CI も検定もしない。

> `exact_auc` だけが例外で、**全スコアをメモリに読む**とコメントに明記してある。

M1 の 0.8382 ± 0.0512 はここから出た。`--epochs 10` で測り直した値

（旧記録 0.8383 はエポック数が書かれていなかった。禁忌34・40）。

4.4 `stream_gbdt.py`（341行）— 有界メモリ GBDT

**「O(1) メモリの制約」と「線形モデルであること」のどちらが 0.0540 の AUC 差を

払っているか**を切り分けるために書かれた。

定数	値
`BINS`	**255**（ヒストグラム配列は 256幅、探索は b = 0..254）
`TREES`	100
`MAX_LEAVES`	**31**（`len(tree.value) < 2 * max_leaves - 1` = 61ノードで強制）
`LEARNING_RATE`	0.1
`MIN_SAMPLES_LEAF`	20
`L2`	**0.0**
`PRE_BINS`	**4096**（ストリームから分位点の境界を導く粗ヒストグラム）
`MAX_DEPTH`	`None` — **宣言されているがファイル内で一度も読まれない**

4.5 `m3_order.py`（449行）— 順序入れ替え〔撤回済み〕

定数	値
`ENGINE`	`{"desc": target/release/engine, "fp": target-fp/release/engine}` — **2ビルド**
`BUDGETS`	**(100, 500, 1000, 5000, 10000)** ＋ `len(pool)` を6番目に追加
`TARGET_COUNTS`	(1, 5, 20)
`RANDOM_REPEATS`	20
`BINDING_KEEP`	**0.25** — 結合評価を通る割合。出力数は `m = max(1, round(budget * 0.25))`
`MAX_TIE_SHARE`	**0.05**
`MIN_DISTINCT_SHARE`	**0.20**

M3 の主張は撤回されている。 `sh()` `stats_from()` `write_parts()` は共通部品として

`screenable.py` が今も使う。**`#lock_0` あり。**

4.6 `transfer.py`（195行）— 転移検証

定数	値
`BOOTSTRAP_ROUNDS`	**20000**（`inclusion.test` と `answerable.main` が**実行時に書き換える**）
`ENGINE`	`--engine` で `globals()["ENGINE"]` ごと差し替え可

基準は実行前に宣言した。ただし 2026-10-09 に**判定を撤回**（数字は有効）。

宣言した基準（CI 下限 > 0.5）が評価件数と交絡していた。

4.7 `inclusion.py`（445行）— 包含関係 → 転移の向き〔支持されず〕

定数	値
`DEFAULT_SOURCE`	`B3DB_classification_feat/features.parquet`
`ASYMMETRY_FACTOR`	**3.0**（この比以上で二次判定が予測を出す）
`SPLIT_SEED`	0
`MIN_SUBSET`	**150** — 「ドメインと両クラスに足りるだけの分子が各辺に要る」
`CONTROL_PAIR`	**`"random_halves"`** — 主たる符号検定から**除外される対照**
`BOUNDARY_MARGIN`	**0.05** — 「CI 下限が 0.5 にこれだけ近いと、判定がブートストラップ回数に左右される」

ソートしてからシャッフルする

```
big.sort(); small.sort()
rng = random.Random(seed); rng.shuffle(big); rng.shuffle(small)
```

> **シャッフルの前にソートする。**順序が dict の順ではなく seed に依存するように。

結果は「反証」ではなく「判定不能」。事前登録は `docs/inclusion_predictions.json`。

4.8 `answerable.py`（256行）— 箱の外は答えられないのか〔否定〕

定数	値
`MIN_SIDE`	**50**（これ未満の辺は測定不能）

差のブートストラップ CI を取る。2群は互いに素なので差が素直に出る。

> 最初は2つの CI の重なりで判定していたが、**それは禁忌44 そのもの**なので差し替えた

> （→ 禁忌54：自分が書いた禁忌を次の設計で踏む）。

結果：covered が有意に良い方向は 12 中 0。

4.9 `coverage.py`（240行）— カバレッジ

定数	値	宣言
`NON_FEATURE_COLUMNS`	`("label", "scaffold", "smiles", "inchikey", "skeleton", "tables")`	—
`BOX_LOW_PERCENTILE`	**1**	**「実行前に固定。CRITERIA 参照」**
`BOX_HIGH_PERCENTILE`	**99**	同
`NEIGHBOUR_PERCENTILE`	**95**	訓練集合内部の leave-one-out 最近傍距離の分位点

指標	定義
box	全記述子が訓練側の 1〜99 パーセンタイル内。**必要条件にすぎない**
neighbour	標準化記述子空間での最寄り訓練点までの距離が、訓練セット内部の最近傍距離分布の **p95 以下**
covered	両方

最近傍基準に変えると CHAMP は 6.2% → 0.6%（10分の1）。箱だけを報告していたのが

誤りだった（禁忌36）。

ただし `answerable.py` が直接検定した結果、membership は AUC を予測しない。

距離の記述としては有効、能力の上限としては書けない。

4.10 `globularity.py`（191行）— eNTRy 第2条件〔このライブラリでは無効〕

定数	値
`ENTRY_MAX_GLOBULARITY`	**0.25**（Richter 2017 の閾値）
`EMBED_SEED`	**0xC0FFEE**（`AllChem.ETKDGv3().randomSeed`。配座生成が再現可能）

定義：原子座標の共分散行列の条件数の逆数。1 が完全な球、0 が平板または線状。

ベンゼン 0、アダマンタン 1。論文の観測値：アンピシリン 0.12、

ペニシリン G 0.17、デオキシニボマイシン 0.02。

ZINC で 300件サンプル中 100% 通過（中央値 0.075、最大 0.247、閾値 0.25）。

適用しても何も変わらない。「適用した」と書くとコードについては真、

screening については偽（禁忌68）。

配座で中央値 0.033、最大 0.236 動く（禁忌69）。

4.11 `m3_models.py`（66行）— 各モデル単体の AUC

> **推論的な統計は無い。**検定もCIもブートストラップも fold 分割も RNG seed も無い。

5. 取り込み〔計測〕

5.0 全体に共通する不変量

	値	なぜ
`PART_ROWS`	**8192**（全取り込みモジュール共通）	1パート1行グループ。**不変条件2**
`MAX_PARSE_FAILURE_RATE`	**0.01**	これを超えたら**データではなく読み手が疑わしい**
`MAX_RADICAL_RATE`	**0.01**	不対電子が残った分子の割合。**方言の不一致のうちエラーを出さない側**

`data/raw/` は読むだけ。生成物は `data/work/`。

5.1 `smiles.py`（136行）— 唯一のパーサ

修復する方言は2種類。どちらも GramNegAccum の表で実際に出た。

定数	値
`_BRACKET_CHARGE_FIRST`	`re.compile(r"\[([A-Z][a-z]?)(@{0,2})([+-]\d*)(@{0,2})(H\d*)\]")`
`ORGANIC_SUBSET`	`frozenset((5, 6, 7, 8, 15, 16, 9, 17, 35, 53))` = B, C, N, O, P, S, F, Cl, Br, I

1. ブラケット内のトークン順序 — `[N+H2]`、`[N+@@H]`。電荷が水素数より前

2. 水素数の未指定 — 既定原子価から再推定

`ORGANIC_SUBSET` に限る理由（禁忌30）：原子価則を金属に当てると、B3DB の

`[Li]` が `[LiH]` に書き換わった。**別の化合物になる。**

修復は数えて出す（禁忌32）。「何件直したか」を summary に出さないと、

修復が効いているのか対象が無いのかが分からない。

`#lock_0` あり。

5.2 `ingest.py`（約140行）— 汎用 TSV/CSV

定数	値
DIALECTS	`.tsv` → `delimiter="\t"`、`.csv` → `delimiter=","`。**両方 `quoting=csv.QUOTE_MINIMAL, strict=True`**
`CHUNK_ROWS`	8192

`QUOTE_NONE` にしてはいけない（禁忌7）。TSV も引用符を解釈するので、

改行を含むレコードが壊れる。`strict=True` は不正な引用を**そのレコードのエラー**に

する — 黙って後続行と結合しない。

`data/raw/` 直下の `.tsv` / `.csv` だけを取る（サブディレクトリは専用の取り込みが扱う）。

走査結果にディレクトリが混ざらないことを `tests/test_contraindications.py` が固定（禁忌13）。

5.3 `ingest_zinc.py`（約160行）— ZINC22 tranche

形式	**タブ区切り2列（SMILES、ZINC id）、ヘッダ無し**
誤り	計画書は「ヘッダあり」と書いていた。**1分子失う**
ディレクトリ名	`H` の数字が**重原子数そのもの**
1トランシェの幅	zinc-22x H29P240-N は **350〜596 Da**

件数を3通りで報告する（禁忌25）：読んだ行数、パースできた分子数、

distinct な InChIKey 骨格数。**これらは違う値で、混同が実際に起きた。**

`MAX_PARSE_FAILURE_RATE = 0.01`。実測は zinc-22x H29P240-N の 144,929分子で **0.000%**。

`parts.json` に `source` / `source_name` / `manifest` / `parts` / `rows_written` /

`lines_read` / `molecules_parsed` / `repairs` / `dropped` を記録。

5.4 `ingest_champ.py`（約170行）— xlsx を直接読む

SpreadsheetML を zip から XML で読む。pandas も openpyxl も使わない。

定数	値
`NS`	`{http://schemas.openxmlformats.org/spreadsheetml/2006/main}`
`SHEETS`	`sheet13.xml` → `wt`（F列）／ `sheet16.xml` → `pmbn_wt`（E）, `porinated_wt`（F）／ `sheet18.xml` → `porinated_wt_check`（E）, `porinated_tolc_ko`（F）
`SMILES_COL`	**`D`**（全シート共通）
`HEADER_ROWS`	**2**
`LABEL_SOURCE`	**`wt`**

> **シートごとにグループ化している理由：値は同一シート内でのみ比較可能。**

5.5 `ingest_gna.py`（約150行）— GramNegAccum

定数	値
`TABLES`	`((1, "smiles"), (2, "smiles"), (3, "smiles"), (4, "mol"))`
`MAX_RADICAL_RATE`	**0.01**

table 5 と 6 には Accum 列が無い。だから取らない。

ラジカル率の実測が決定的だった。

```
py/smiles.py あり : 0.000%
py/smiles.py なし : 38.2%
```

38.2% の分子が不対電子を持ったまま通っていた。`[N+]`が「水素0個の窒素カチオン」と読まれ、一級アミンのパターンに当たらない。**エラーは出ない。記述子が静かに間違う**（禁忌27）。

件数の内訳が 368 / 234 / 217 / **197** と変わるのがこのデータ（禁忌25）。

5.6 `ingest_imi.py`（約130行）— IMI-COMBINE

定数	値
`SRC`	`data/raw/imi_combine/data/processed/combined_bioassay_data.tsv`
`GP`	**`gram-positive_label`** — 結合の代理
`GN`	**`gram-negative_label`** — true hit（結合 AND 侵入）

両ラベルを持つ行だけを残すので 22,051行。全ソースは 74,202行。

同じラベルの件数がスコープで動く（禁忌81）：gram-positive の active は全ソースで 17,530、両ラベル保有で **8,601**。

5.7 `descriptors.py`（約200行）— 9記述子

```
FEATURES = ("mw", "logp", "tpsa", "hbd", "hba", "rotb",
            "aromatic_rings", "basic_nitrogen", "fsp3")
```

この順序で固定。`engine/src/carry.rs`が`F = 9`として鏡像を持つ。

`BASIC_NITROGEN`の SMARTS は`knobs.py`と同一パターン。コメントに経緯がある：

> **形式電荷を最初に試したが、CHAMP 全体で恒等的に 0 で、信号を一切持たなかった。**

> eNTRy 自身の一級アミンパターンも CHAMP 全体で 0。だからこの広いパターンが

> 「イオン化可能窒素」の最も近い使える代替。

定数	値
`MAX_PARSE_FAILURE_RATE`	0.01
`MAX_RADICAL_RATE`	0.01。**実測：CHAMP / IMI / GNA が 0.000%、B3DB が 0.013%、`smiles.py`無しの GNA が 38.2%。1% が2つの場合を余裕を持って分ける**
`SPLITS` / `SPLIT_FRACTIONS`	**定義されているが本ファイル内で参照されていない。**分割は`folds.py`の仕事

`#lock_0` あり。エンジンがやってはいけない仕事（SMILES を読む、ファイル全体にわたって記述子を計算する）をここが担う。

5.8 `fingerprints.py`（約70行）— ECFP8

定数	値
`FP_BITS`	**2048**。`engine/src/carry.rs` の `F`（fingerprint ビルド）と一致必須
`FP_RADIUS`	**4** = **ECFP8**。「このデータの公表ベースラインが使う半径」
`MAX_RADICAL_RATE`	0.01

半径4は半径2より部分構造が大きくビットが疎なので、同じ Tanimoto 0.4 が ECFP4 よりかなり厳しい。適用範囲の中央値 0.2 はこれで説明が付く。

5.9 `fingerprint_library.py`（約110行）— ライブラリ側の指紋

`PART_ROWS = 8192` が**パート行数・numpy ブロック高・`row_group_size` の3つを兼ねる。**

6. 降格済み〔降格〕

ファイル	内容
`candidates.py`	候補生成。SMARTS と InChIKey 重複排除。**`#lock_0` あり**
`sequential.py`	逐次スクリーニング
`screen.py`	スクリーニング

3本とも `--emit-molecules` なしでは止まる。

`tests/test_request.py` が、**製品経路のどのモジュールもこれらを import しない**ことを固定している。

7. テスト（124本）

ファイル	本数	守るもの
`test_memory_invariant.py`	3	**不変条件。**2軸（データ量・バルクサイズ）。`OCCAM_FACTORS` で振る
`test_smiles.py`	11	SMILES の方言・修復・no-op 確認
`test_descriptors.py`	4	記述子、RSS のスケール平坦性、実データのパース失敗率
`test_folds.py`	5	グループが fold をまたがない、Butina の上限拒否
`test_ingest.py`	2	引用符つきレコード、破棄理由の計数
`test_invariants_locked.py`	6	KeepOut の存在・ロック9箇所・改変検出・同梱コピーのハッシュ照合・ライセンス
`test_render.py`	26	表示層。**文言中の数字と registry の照合**
`test_request.py`	28	パーサの拒否、引退、障壁、降格モジュールへの依存
`test_no_duplicated_counts.py`	6	**同じ件数が2つのリテラルに書かれていたら落ちる**
`test_contraindications.py`	29	禁忌10項目の assertion 化、1セット逐次、スコープ一致
Rust	11	carry 固定長、バルク1つ、標準化定数、AUC、シグモイド有界、top-k

8. 併読

`docs/final_report.md`	**最終報告書（本書の対）。何を establish したか**
`docs/HANDOFF.md`	引継ぎ。§4 確定値、§7 禁忌86項目
`docs/prompt_format.md`	プロンプトの形式
`docs/overbuilt.md`	過剰実装の棚卸し